AI security micropayments on Arc

LaunchShield Swarm

AI security is already a swarm of tiny actions.

Billing and proving each action is the missing layer.

One audit becomes dozens of atomic security tool calls.

LaunchShield turns those calls into a live execution feed, a live billing feed, and a verifiable evidence trail.

63

paid invocations

in the preset stress run

$0.001 - $0.008 per call

Core proof

billing row + result + settlement reference

Repo scans | Dependency checks | Browser probes | Deep LLM review | Fix suggestions

The execution model already changed. The payment model did not.

AI security work creates many high-signal actions with very low value per call.

1 audit

40-60

atomic tool calls

Each call matters. Each call is too cheap for legacy settlement.

Sub-cent value

Many tasks cost less than one cent.

High trust requirement

Security buyers want proof and provenance.

Bad legacy fit

Card rails and high-gas chains crush margin.

Flat pricing hides execution quality, hides cost structure, and hides proof of delivery.

Micropayments turn AI work from a black box into a priced workflow

When each invocation has a price, a result, and a receipt, the workflow becomes commercially legible.

01
Invocation
Which tool ran against which target.

02
Price
A stablecoin-denominated micro-charge.

03
Result
A concrete finding, proof, or artifact.

04
Receipt
A settlement reference that can be checked.

Why it matters

Buyers see what happened. Builders see where value is created. Security teams get proof instead of a black-box summary.

What LaunchShield Swarm does

One AI security audit is broken into priced micro-tasks and rendered live.

A single run includes

- Repository file scanning
- Dependency vulnerability lookups
- Live site probing
- Deep LLM review of high-risk findings
- Verification and fix suggestions

Live product loop

1. **Trigger a run**
2. Fan out into atomic tool calls
3. Stream execution + billing together
4. Surface findings and profitability in one UI
5. Attach payment references and evidence

Atomic tool calls

Live billing waterfall

Evidence

System architecture

A provider-based orchestration layer turns AI security work into billable, observable execution.

Frontend

FastAPI web app | templated UI | SSE event stream

Core runtime

Orchestrator | pricing engine | run storage + profitability

Provider adapters

GitHub / repo source | browser + LLM + AIsa | Arc / Circle

Mock-first by default. Real providers can be enabled one by one. Full repository scan mode is supported.

One run becomes a live waterfall of paid actions

The billing feed is product proof, not decoration.

Preset stress run

```
22:15:37   file_scan       launchshield/app.py
0.001 USDC
22:15:42   dep_lookup      fastapi>=0.110
0.001 USDC
22:15:49   site_probe      target / headers
0.001 USDC
22:15:55   deep_analysis   dangerous_eval
0.008 USDC
22:16:03   aisa_verify     hardcoded_secret
0.001 USDC
22:16:12   fix_suggestion  repo_scan.py:49
0.001 USDC
```

63
paid tool invocations

$0.001-$0.008
per call pricing

Live refs
each row can carry a settlement reference

Cost and provenance stay visible at tool-call granularity.

High-signal outputs, not vague summaries

The run produces concrete findings, deeper review, and fix guidance.

High
Hardcoded secret exposure
Credential material in source indicates unsafe secret handling.

High
dangerous_eval in repo_scan.py:49
eval() on untrusted input creates a code injection path.

High
Missing security headers
Missing CSP / HSTS / X-Frame-Options weakens browser hardening.

Verification path

Rule engine
Static heuristics flag high-confidence issues.

Deep analysis
OpenAI-compatible review adds context and severity detail.

Fix suggestion
Actionable remediation closes the loop.

Finding cards can be traced back to per-call billing rows.

Proof of settlement on Circle and Arc

Billing row -> payment reference -> Circle console -> Arc explorer.

Billing row -> **Payment ref** -> **Circle console** -> **Arc explorer**

Circle receipt panel

Evidence placeholder

Settlement receipt / console verification / amount + reference

Arc explorer panel

Evidence placeholder

On-chain transaction proof / tx hash / explorer deep link

Why Arc + stablecoin settlement matters economically

Traditional gas-heavy settlement destroys the margin of high-frequency AI security workflows.

Example workflow service cost: 0.152

Traditional EVM

$3.15

estimated gas overhead

Same workflow. Margin gets crushed before the product scales.

Arc + Circle

0.152 USDC

settled workflow cost

Fair usage-based billing stays commercially viable.

- High frequency
- Low ticket size
- Many repeated actions
- Margin-sensitive

Who needs this first

The first wedge is any team that already feels the pain of opaque AI cost and opaque AI provenance.

AppSec teams
High trust requirements. Clear security outputs.

Security consultancies
Per-call proof helps justify delivered work.

AI agent platforms
Need usage-based monetization primitives.

Developer tool marketplaces
Need transparent pricing for agent actions.

"I cannot ship an agent that costs more to settle than it charges."

Roadmap

This MVP proves the workflow. The roadmap expands precision, integrations, and deployment depth.

Near term

- Multi-tenant runs and team workspaces
- Richer scanners and better findings
- Deeper AIsa integration
- More real-provider hardening

Mid term

- Agent marketplace integrations
- Repeatable production billing flows
- Better dashboards for many tiny settlements

Long term

- Arc mainnet launch
- Production-grade usage-based security workflows
- Agent work priced, settled, and audited natively

Closing

LaunchShield Swarm

Billable. Traceable. Verifiable AI security workflows.

Arc and Circle make the workflow economically viable.

Billable
AI security at tool-call granularity

Traceable
AI security at tool-call granularity

Verifiable
AI security at tool-call granularity

Product thesis
Any AI workflow that becomes many tiny valuable actions will need native micropayment rails.

GitHub: github.com/Abdullahccgdq/launchshield
Demo: launchshield-swarm.fly.dev | Team: Red Snail